INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming Lab
[PCCCS495]

Student Attendance Management System

Name: -Suranjan Kuila

Semester: -IV

Class Roll: - 63

Enrolment Number: - 12024052004060

Date: - 29/03/2026

Instructor name: - Dr.Susovan Jana, DR.Soumadip Biswas, Dr.Swagatam Basu

Department of Information Technology

2026

2. Problem Statement:

Managing student attendance manually is time-consuming and prone to errors. Traditional methods often lack efficiency in tracking attendance records, calculating percentages, and generating reports.

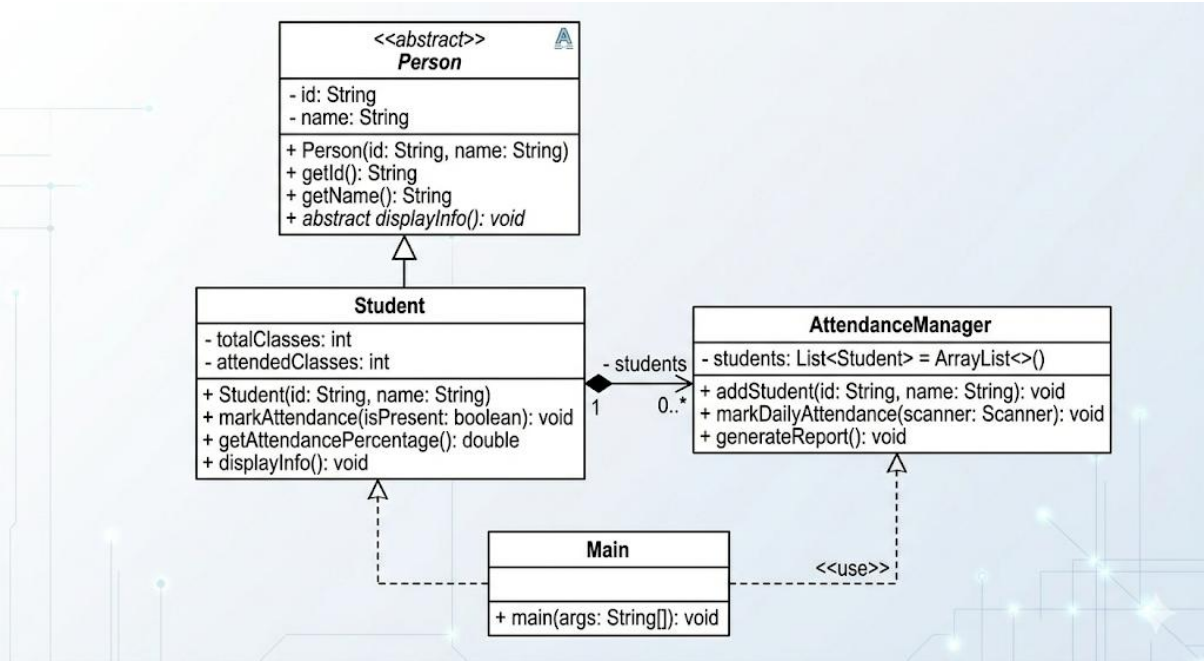
This project aims to develop a **console-based Student Attendance Management System** using Java. The system allows users to:

- Add new students
- Mark daily attendance
- Generate attendance reports

The goal is to create a structured, scalable, and error-resistant system using Object-Oriented Programming (OOP) principles.

3. System Design:

3.1 UML Class Diagram (Textual Representation):



3.2 Description of Major Classes:

i. Person (Abstract Class)

- Base class representing a general person.
- Contains common attributes: id and name.
- Defines abstract method displayInfo().

ii. Student (Derived Class)

- Extends Person.
- Tracks attendance data (totalClasses, attendedClasses).
- Implements attendance calculation logic.

iii. AttendanceManager

- Manages a collection of students.
- Handles adding students, marking attendance, and generating reports.

iv. Main Class

- Entry point of the application.
- Provides menu-driven interaction with the user.

3.4 Interaction Flow:

1. User starts the program
2. Menu is displayed
3. User selects an option:
    - Add student → stored in list
    - Mark attendance → loop through students
    - Generate report → display all records
4. System continues until user exits

4. OOP Concepts Demonstrated:

4.1 Abstraction

Where used: Person abstract class

Why used: To define a common structure while hiding implementation details.

Code Snippet:

```
abstract class Person {
   private String id;
   private String name;
   public abstract void displayInfo();
}
```

Explanation:
The abstract class ensures that every subclass must implement displayInfo() while sharing common attributes.

4.2 Inheritance

Where used: Student extends Person

Why used: To reuse code and establish hierarchy.

Code Snippet:

```
class Student extends Person {
    public Student(String id, String name) {
        super(id, name);
    }
}
```

Explanation:
The Student class inherits properties of Person, reducing redundancy.

4.3 Encapsulation

Where used: Private fields with getters

Why used: To protect data and control access.

Code Snippet:

```
private String id;

public String getId() {
    return id;
}
```

Explanation:
Data is hidden from direct access and accessed through methods.

4.4 Polymorphism

Where used: Method overriding (displayInfo())

Why used: To allow different implementations of the same method.

Code Snippet:

```
@Override
public void displayInfo() {
```

```
    System.out.println("ID: " + getId());
}
```

Explanation:
Student provides its own version of displayInfo().

---

5. Implementation Highlights:

5.1 Adding Students with Validation

```
public void addStudent(String id, String name) {
    if (id.isEmpty() || name.isEmpty()) {
        throw new IllegalArgumentException("ID or Name cannot be empty!");
    }
    students.add(new Student(id, name));
}
```

Justification:
Ensures no invalid data is stored.

5.2 Attendance Marking Logic

```
student.markAttendance(input.equals("y"));
```

Justification:
Simplifies attendance marking using boolean logic.

5.3 Attendance Percentage Calculation

```
return (attendedClasses * 100.0) / totalClasses;
```

Justification:
Ensures accurate percentage calculation using floating-point arithmetic.

5.4 Input Validation Loop

```
if (!input.equals("y") && !input.equals("n")) {
    throw new Exception("Invalid input!");
}
```

Justification:
Prevents incorrect user input during attendance marking.

6. Testing & Error Handling:

6.1 Test Cases

Test Case	Input	Expected Output
Add student	Valid ID & name	Student added
Empty name	""	Error message
Mark attendance	y/n	Recorded correctly
Invalid input	x	Error message

6.2 Edge Cases Handled

- No students in system
- Division by zero (attendance)
- Empty input fields
- Invalid menu choices

6.3 Failure Scenarios

- Invalid numeric input → handled using NumberFormatException
- Wrong attendance input → handled using custom exception
- Empty student details → handled using IllegalArgumentException

7. Git Workflow:

Commits

History for term-ii-project-submission-suranjan2004 / src on main

All users | All time

Commits on Mar 29, 2026

Detect error in code and solve debugging
suranjan2004 committed 8 hours ago — 106f19c

Handle the errrors in code
suranjan2004 authored 9 hours ago — Verified — c7e4990

Commits on Mar 28, 2026

Add switch case to take the user input
suranjan2004 authored 13 hours ago — Verified — 8579b7f

Commits on Mar 27, 2026

Add generateReport class that show the attendance report
suranjan2004 authored 2 days ago — Verified — f14f547

Commits on Mar 26, 2026

Exception handaling for AttendanceManager class.
suranjan2004 authored 3 days ago — Verified — 303c0b8

Commits on Mar 25, 2026

add AttendanceManager class to add and manage student attendance
suranjan2004 authored 4 days ago — Verified — 4a8f181

Commits on Mar 24, 2026

Update .keep
suranjan2004 authored 5 days ago — Verified — 91cccc6

Student class inherit from person class
suranjan2004 committed 5 days ago — fb35210

Add person class
suranjan2004 committed 5 days ago — 50a74c8

create person class
suranjan2004 authored 5 days ago — Verified — 9a5a8c1

Commits on Mar 2, 2026

Initial commit
github-classroom[bot] authored 27 days ago — Verified — 59939a0

End of commit history for this file

8. Conclusion & Future Scope:

The Student Attendance System effectively applies OOP concepts to manage records efficiently through a simple console interface. It ensures data validation, structured design, and user interaction. Future improvements may include a graphical user interface, database integration for persistent storage, multi-user support, and advanced analytics like attendance trends and automated report generation.

Appendix:

Approved Email: Suranjan.Kuila2024@iem.edu.in